

aws-s3-files-performance-test-cdk



License MIT

Benchmark tool that evaluates four AWS file-storage access patterns from an ECS Fargate task across three dimensions: **performance**, **operations**, and **cost**.

Benchmark Cases

#	Storage	Access Pattern	Key Question
1	S3 Standard	Download to <code>/tmp</code> → local R/W	Baseline throughput with multi-AZ durability
2	S3 One Zone-IA	Download to <code>/tmp</code> → local R/W	Cost saving vs. Standard; single-AZ trade-off
3	EFS	Direct mount R/W (NFS)	POSIX semantics; shared file system latency
4	S3 Express One Zone	Direct <code>GetObject</code> / <code>PutObject</code>	Low-latency object API without local copy

Each case runs **10 iterations** on a **100 MB** file, recording per-iteration download time, read throughput, write throughput, P95/P99 latency, and error rate.

Architecture

```
graph TD
    subgraph VPC
        ECS[ECS Fargate Task\n2 vCPU / 4 GB]
        EFS[EFS\nElastic throughput]
        ECS -- NFS mount --> EFS
    end

    ECS -- GetObject / PutObject --> S3STD[S3 Standard]
    ECS -- GetObject / PutObject --> S3OZ[S3 One Zone-IA]
    ECS -- GetObject / PutObject --> S3EX[S3 Express One Zone\nDirectory Bucket]

    ECS -- results --> SSM[SSM Parameter Store\n/benchmark/results]
    ECS -- logs --> CWL[CloudWatch Logs\n/ecs/s3files-benchmark]
```

Evaluation Dimensions

Performance

- **Download time** – time to transfer 100 MB from S3 to the task (Cases 1 & 2 only)
- **Read throughput** – MB/s reading the 100 MB file
- **Write throughput** – MB/s writing the 100 MB file
- **P95 / P99 latency** – tail latency across 10 iterations

Operations

- **Mount management** – EFS requires NFS mount lifecycle; S3 cases use SDK calls
- **POSIX compatibility** – EFS supports append, seek, locks; S3 does not
- **Availability** – Standard S3 / EFS are multi-AZ; OneZone variants are single-AZ

Cost (approximate, Tokyo region)

Storage	Per GB-month	Per GET	Per PUT	Notes
S3 Standard	\$0.025	\$0.00037/1k	\$0.0047/1k	Multi-AZ
S3 One Zone-IA	\$0.019	\$0.01/GB retrieval	\$0.01/1k	30-day min
S3 Express One Zone	\$0.16	\$0.002/1k	\$0.005/1k	Single-digit ms
EFS Elastic	\$0.043	—	—	+ \$0.06/GB transferred

Prerequisites

Tool	Version
Python	3.12+
Node.js	20+
AWS CDK CLI	v2 (<code>npm install -g aws-cdk</code>)
Docker	24+
AWS CLI	v2, configured with appropriate permissions

The deploying IAM principal needs permissions for: CloudFormation, EC2, ECS, EFS, S3, S3Express, IAM, Lambda, SSM, CloudWatch Logs, ECR.

Quick Start

```
# 1. Clone and install CDK dependencies
git clone https://github.com/your-org/aws-s3-files-performance-test-cdk.git
cd aws-s3-files-performance-test-cdk

python -m venv .venv
## source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 2. Bootstrap CDK (first time per account/region)
```

```
cdk bootstrap --region ap-northeast-1

# 3. Deploy (builds and pushes Docker image automatically)
cdk deploy --region ap-northeast-1

# 4. Run the benchmark and print results (~10 min)
bash scripts/run_task.sh ap-northeast-1
```

Example output

```
=====
BENCHMARK RESULTS
=====
```

Case	AvgRead	AvgWrite	P95R	P95W
DL avg				

Case1 Standard-S3 → /tmp	1024.3MB/s	980.1MB/s	98ms	102ms
850ms				
Case2 S3-OneZone-IA → /tmp	1018.7MB/s	975.4MB/s	101ms	105ms
860ms				
Case3 EFS-direct	112.4MB/s	98.6MB/s	910ms	980ms
0ms				
Case4 S3-Express-OneZone-direct	320.5MB/s	295.8MB/s	315ms	340ms
0ms				

Results vary by instance placement, VPC routing, and EFS throughput mode.

Project Structure

```
.
├── benchmark/                # Containerised benchmark application
│   ├── benchmark.py          # Main benchmark script (4 cases)
│   ├── Dockerfile
│   ├── requirements.txt
│   └── .dockerignore
├── infra/                    # AWS CDK stack
│   ├── __init__.py
│   └── stack.py
├── scripts/
│   └── run_task.sh           # Launch task & fetch results
├── .github/
│   ├── ISSUE_TEMPLATE/
│   └── workflows/
│       └── lint.yml
├── app.py                    # CDK entry point
├── cdk.json
├── requirements.txt          # CDK dependencies
└── CHANGELOG.md
```

```
└─ CONTRIBUTING.md
└─ LICENSE
```

Customisation

Variable	Default	Description
<code>FILE_SIZE_MB</code>	<code>100</code>	Test file size in MB (edit <code>benchmark/benchmark.py</code>)
<code>ITERATIONS</code>	<code>10</code>	Number of R/W iterations per case
Task CPU	<code>2048</code>	Fargate CPU units (edit <code>infra/stack.py</code>)
Task Memory	<code>4096</code>	Fargate memory MiB

Cleanup

```
cdk destroy --region ap-northeast-1
```

This removes all provisioned resources. Note: EFS data and S3 objects are deleted automatically because `RemovalPolicy.DESTROY` is set.

Contributing

See [CONTRIBUTING.md](#) for development setup, code style, and the pull request process.

License

[MIT](#)